

Simone Rizzo · 2°
CEO @ Inferentia | AI Engineer
22.4.2026

link to Simone
Rizzo's
video

+ Segui

OpenAI releases a new Open Source model and it is also useful for privacy!!

Yes, it really happened... and this time they got it right!

The problem is always the same: we want to use the most powerful models (GPT, Claude, Gemini, GLM, MiniMAX, Kimi, etc.) but we are terrified of sending sensitive data to the cloud. Names, emails, phone numbers, addresses... a nightmare for compliance! 🤯

Well, OpenAI has just "dropped" Privacy Filter, a model designed specifically to anonymize your prompts and documents BEFORE feeding them to AI.

In practice, it is the "bouncer" for your privacy, standing at your front door (that is, on your local servers). 🛡️❤️

Here is why, in my opinion, it is fantastic:

- **100% Local:** It runs on your machine. It does not need to call any external API to clean the data.

- **Ultra-lightweight:** It works very well on both GPU and CPU. You do not need a supercomputer to run it; a modern laptop is more than enough! 🖥️

- **Zero Leak:** It identifies and removes (or replaces) sensitive data (PII), transforming a "dangerous" document into anonymous text ready for the cloud.

- **Free and Open:** It is open-source. You download it, integrate it into your pipeline, and that is it.

🗨️ **My advice:** If you work in a company or have a startup where data management is sensitive, this is the missing piece of the puzzle. You can finally unlock the use of cloud models without risking a huge fine or a leak of sensitive data.

I am already testing it for some workflows and I have to say that the accuracy in catching sensitive entities is truly remarkable. This time OpenAI has given the community a great gift!

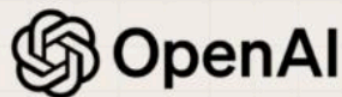
Find everything here on the official repo: 🗄️ <https://lnkd.in/eBj8d26y>

Privacy Filter predicts spans across eight categories:

- private_person
- private_address
- private_email
- private_phone
- private_url
- private_date
- account_number
- secret

OpenAI Privacy Filter is a bidirectional^{1/7} token-classification model for **personally identifiable information (PII) detection and masking in text**. It is intended for high-throughput data **sanitization** workflows where teams need a model that they can **run on-premises that is fast, context-aware, and**

link to my Word document
explaining how to install it



New open-source model

Privacy Filter: your exposed data *protected before the prompt.*

Original text

Customer Mario Rossi,
born on 12/04/1984,
email m.rossi@acme.it,
IBAN IT60 X054...

Protected text

Customer [PERSON],
born on [DATE],
email [EMAIL],
IBAN [ACCOUNT]

PRIVACY FILTER

link to the OpenAI GitHub repo
→ "How to use" section

Apache 2.0 · Free

CPU · GPU · Browser

128,000 context tokens

OpenAI **Privacy Filter** is a tool designed to **intercept, recognize and mask personal or potentially sensitive information** present in text before it is used in an automated processing workflow, for example in an artificial intelligence pipeline, in a RAG system, in a training dataset, in an application log or in a prompt intended for a language model. Its main purpose is to **reduce the risk** that identifying data, such as names, addresses, emails, dates of birth, phone numbers or other personal references, are transmitted or stored unnecessarily. Internally, the filter is not limited to a simple mechanical search for words or patterns, but analyzes the text in its context, identifies the **portions that may represent personal data and replaces them with labels, masking or redacted forms**. In this sense, Privacy Filter is positioned among preventive data-protection tools: it does not change the logic of the document nor necessarily remove its general meaning, but acts on the elements that could allow a person to be identified or confidential information to be disclosed. However, it should be regarded as a **risk-mitigation** tool, not as an absolute guarantee of anonymization, because automatic recognition can always produce **omissions or excessive masking**, especially when personal data is **implicit, ambiguous or context-dependent**.

Summary table

Test	Device	Rows	Time (seconds)	Readable time	Throughput rows/sec	Result
test_1000.txt	CPU	1,000	913.995	15 min 14 s	1.09	OK
test_1000.txt	GPU	1,000	18.931	19 s	52.82	OK
test_10000.txt	GPU	10,000	135.994	2 min 16 s	73.53	OK
test_50000.txt	GPU	50,000	669.741	11 min 10 s	74.66	OK
test_50000.txt	CPU	50,000	45,068.726	12 h 31 min 9 s	1.11	OK

run only on GPU

CPU/GPU comparison

Rows	CPU	GPU	GPU speedup
1,000	913.995 s	18.931 s	about 48.3x
50,000	45,068.726 s	669.741 s	about 67.3x

returncode = 0
means that the command completed without errors.

Technical comment

The behavior is consistent. The CPU processes about **1.1 rows per second**, both in the 1,000-row test and in the 50,000-row test. This indicates fairly linear scalability, but with very low throughput.

The GPU, on the other hand, after the initial loading and initialization cost, stabilizes at around **74 rows per second** on larger files. In fact:

GPU 10000 approx. 73.5 rows/sec

GPU 50000 approx. 74.7 rows/sec

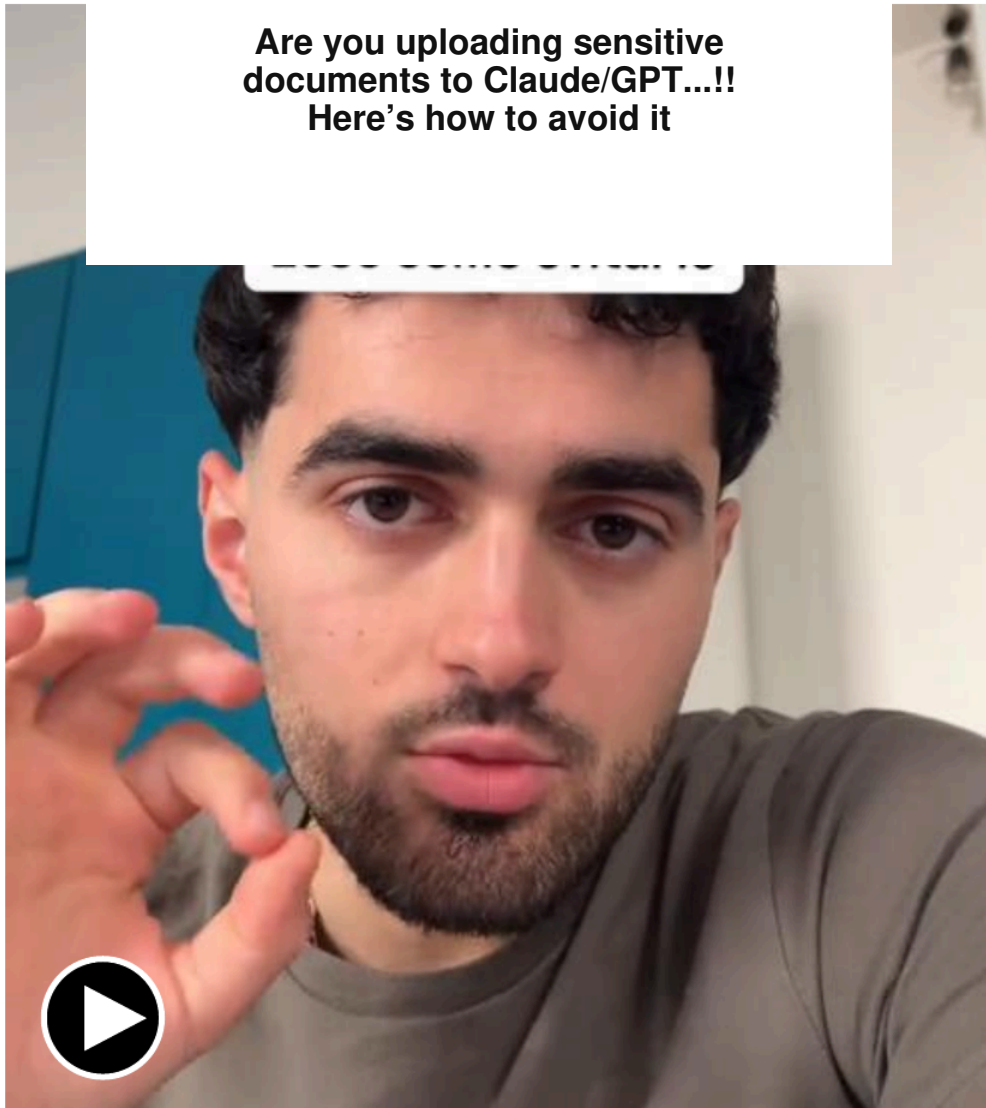
This means that the GPU works much more efficiently as volume increases. The 1,000-row test is penalized by the initial overhead: loading Python, the model, PyTorch, CUDA, reading the file and writing the output. With 10,000 and 50,000 records, on the other hand, the overhead is absorbed and the actual throughput emerges more clearly.

CPU: suitable only for small tests or emergencies

GPU: required for real workloads

olsixhindoli.ai

Are you uploading sensitive documents to Claude/GPT...!!
Here's how to avoid it



Protect sensitive data with AI

Discover how AI can mask sensitive data in your documents. Protect your privacy and comply with regulations. Read now!
#Privacy #DataSecurity

Keywords: ["AI data masking", "sensitive data protection", "AI document security", "legal privacy", "GDPR compliance", "AI privacy tools", "automatic masking", "sensitive data management", "cybersecurity solutions", "information protection"]

```
(privacy-filter) C:\Users\Utente\privacy-filter-openaI>opf eval examples/data/sample_eval_five_examples.jsonl
```

```
summary:
```

field	value
examples	5
tokens	145
windows	5
window_tokens	145
padded_window_tokens	145
elapsed_s	27.0687
tokens_per_s	5.3567
window_tokens_per_s	5.3567
padded_window_tokens_per_s	5.3567
eval_mode	typed
inference_tokens_per_second	3.8387
loss	0.4305
token_accuracy	0.9448

OpenAI example - 1/2

```
detection_metrics:
```

metric	value
detection.precision	0.9697
detection.recall	1.0000
detection.f1	0.9846
detection.f2	0.9938
detection.span.precision	0.9375
detection.span.recall	0.8571
detection.span.f1	0.8955
detection.span.f2	0.8721

```
timings:
```

stage	seconds
startup	3.5524s
window_stage_wall	23.3929s
input_pipeline	0.0000s
model_forward	20.7888s
score_stitch	16.9076s
metrics_prepare	0.0439s
decode	0.0764s
prediction_assembly	0.0001s
metrics_accuracy	0.0001s
metrics_finalize	0.0001s
metrics_total	0.1207s
prediction_write	0.0000s
eval_core_total	23.5137s
eval_total_with_write	23.5137s
overall_wall	27.0764s

```
throughput:
```

metric	tokens_per_second
inference_only_tokens_per_second	3.8387
model_forward_tokens_per_second	6.9749
eval_total_tokens_per_second	6.1666

```
throughput_note: inference_only_tokens_per_second uses model_forward + score_stitch + decode + prediction_assembly.
```

What this command is doing

It is:

- taking 5 annotated examples (ground truth)
- running the model
- comparing predictions vs ground truth
- producing quality + performance metrics

summary section

OpenAI example - 2/2

```
examples: 5
tokens: 145
```

- 5 sentences analyzed
- 145 total tokens

```
tokens_per_s: 5.3567
inference_tokens_per_second: 3.8387
```

speed:

- ~5 tokens/sec overall
- ~3.8 tokens/sec inference only

Very slow - normal:

- small GPU (GT 1030)
- or partial CPU fallback

```
loss: 0.4305
token_accuracy: 0.9448
```

- loss - model's average error (lower = better)
- accuracy ~94.5% - good

detection_metrics section (the most important)

This is the actual quality of the system.

Precision

```
detection.precision: 0.9697
```

when the model flags sensitive data

- it is correct in 97% of cases
- few false positives

Recall

```
detection.recall: 1.0000
```

finds all sensitive data

- zero false negatives

F1 score

```
detection.f1: 0.9846
```

balance between precision and recall

- very high value - very good model

F2 score

```
detection.f2: 0.9938
```

gives more weight to recall

- consistent with recall = 1.0

Span metrics (stricter)

These evaluate whether the model finds exactly the correct span of text

```
detection.span.precision: 0.9275
detection.span.recall: 0.8571
detection.span.f1: 0.8905
```

lower = normal

Why:

- it is not enough to find "a date"
- it must find exactly the correct characters